RolloutOS: A Failure-Aware Rollout Runtime for Reproducible and Resource-Efficient Agent Reinforcement Learning

Anonymous Authors[1]

Abstract

Long-horizon language-agent reinforcement learning is increasingly limited by rollout production rather than only by optimizer throughput. Rollouts interact with browsers, tools, sandboxes, memory, and external services, so expensive samples frequently fail because of runtime loops, invalid actions, timeouts, stale environments, context exhaustion, or unreplayable state drift. We present RolloutOS, a rollout runtime that treats agent-environment interaction as a managed systems workload: sessions have explicit budgets and lifecycle state, failures have operational semantics, traces are recorded as replayable event streams, environments are managed through health and contamination checks, and scheduling optimizes useful trajectories per resource-hour. The key idea is to convert failed rollouts from opaque exceptions into typed, replayable, and schedulable systems artifacts. We evaluate RolloutOS with a controllable failure benchmark and agent workloads spanning browser, tool-use, and code-sandbox environments, measuring useful rollout yield, replay success, failure attribution, tail latency, and environment cost.

1 Introduction

Large language model agents are moving from single-turn preference optimization toward reinforcement learning (RL) in multi-turn, stateful environments. Recent Agent RL systems generate training data by repeatedly running policies against browsers, tool APIs, code sandboxes, web tasks, databases, and other external systems. In such workloads, the rollout layer is no longer a passive data loader. It is a distributed systems substrate that allocates environments, enforces budgets, validates actions, handles failures, records provenance, and decides which partial trajectories reach the optimizer.

Existing Agent RL pipelines often treat this substrate as incidental engineering. The common implementation pattern is a task loop with a timeout, a message transcript, and a binary success/failure flag. This is insufficient for systems-scale rollout production. A failed 30-turn episode may have consumed scarce environment time, generated useful pre-failure behavior, corrupted a browser session, hit an external rate limit, or exposed a policy-attributable invalid action. Discarding it as a generic failed sample loses information that matters for scheduling, debugging, reproducibility, and training data quality.

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

This paper argues that long-horizon Agent RL needs a rollout operating layer. The layer should manage sessions like processes: each rollout has resource ownership, lifecycle state, budgets, event logs, failure state, replay state, and cleanup semantics. It should distinguish policy failures from tool, scheduler, and environment failures. It should provide event-sourced provenance rather than only conversational messages. It should manage environment reuse without silently contaminating future samples. Finally, it should schedule work to maximize useful learning signal under resource constraints, not merely raw request throughput.

We present RolloutOS, a failure-aware rollout runtime for Agent RL workloads. RolloutOS introduces four systems mechanisms. First, it defines a typed failure semantics layer that attaches operational meaning to runtime failures: whether a failure is policy-attributable, retryable, salvageable, training-relevant, or requires environment reset. Second, it records rollouts as append-only event streams containing model, tool, observation, reward, failure, and environment provenance. Third, it manages session and environment lifecycle using watchdogs, health checks, snapshot metadata, contamination checks, and cleanup policies. Fourth, it uses failure-aware scheduling to prioritize rollout requests by expected useful output rather than capacity alone.

This is a systems paper, not a new RL algorithm. RolloutOS is designed to sit beneath policy optimizers such as PPO, GRPO, RLOO, or other Agent RL objectives. The runtime changes how rollouts are produced, validated, replayed, and scheduled; it does not require changing the optimizer.

This paper makes the following contributions:

- We characterize long-horizon Agent RL rollouts as a systems workload whose dominant waste modes include late failures, loops, timeouts, unreplayable traces, and contaminated environments.
- We design RolloutOS, a rollout runtime with typed failure semantics, event-sourced trace provenance, session watchdogs, environment lifecycle management, replay support, and useful-yield scheduling.
- We implement a Python prototype with explicit adapters for tools and environments, JSONL event logs, replay reports, failure-aware scheduling, and command-line inspection utilities.
- We propose and evaluate a controllable FailureBench workload together with browser, web-shopping, and code-sandbox agent workloads, using systems metrics such as useful trajectories/hour, failed rollout cost, replay success, contamination-induced failure, and tail latency.

2 Background and Motivation

2.1 Rollout Production in Agent RL

Agent RL differs from conventional RLHF-style data generation in three ways. First, the environment is long-horizon and partially observable. Second, state is external to the model: browser DOMs, filesystem trees, databases, memory stores, API quotas, and containers can all mutate during an episode. Third, a single rollout often contains many model calls and tool calls, so failures can occur long after substantial cost has already been incurred.

These properties make rollout production a systems bottleneck. The training loop may request a group of rollouts for the same prompt or task, but the useful output depends on runtime behavior: whether workers remain alive, environments reset correctly, tool schemas are respected, model context stays within budget, and traces can be converted into valid training examples.

2.2 Why Message Logs Are Not Enough

Most agent frameworks can save a transcript of user, assistant, and tool messages. For RL systems, this transcript is incomplete. It does not reliably encode the model version, tokenizer, loss mask, log probabilities, tool schema version, environment snapshot, timeout, resource lease, or failure attribution. It also does not explain whether the failed sample should be retried, discarded, salvaged, or used as a negative training signal.

RolloutOS therefore uses an event stream as the primary record. Messages are represented as payloads inside typed events, not as the whole trace. This design allows downstream replay, auditing, scheduling, and training conversion to reason about the same execution artifact.

3 Characterization Study

[TODO: Populate this section with measurements from raw baseline runs.] We first characterize rollout waste in baseline agent loops. The goal is not to show that any single task is difficult, but to expose systems failure modes that are obscured by binary task success metrics.

Workloads. We use four workloads. FailureBench is a synthetic controllable benchmark with oracle failure labels. MiniWoB++ exercises browser state, DOM interaction, reset latency, stale pages, and repeated actions. WebShop or AgentBench web shopping exercises tool-style web interaction, invalid search loops, and sparse rewards. A SWE-bench Verified debug subset exercises shell tools, filesystem mutation, setup failures, and long-running commands.

Metrics. We report completed trajectories/hour, useful trajectories/hour, successful trajectories/hour, failed rollout cost, environment-hours/success, zombie session rate, rollout tail latency, replay success rate, and percentage of failures with actionable attribution. For synthetic workloads, we also report failure detection precision, recall, macro F1, and turns wasted after the oracle failure point.

Expected findings. Our preliminary prototype motivates three hypotheses. First, many failures are detected late by timeout-only baselines, wasting turns and environment time. Second, message-only traces are often insufficient to classify replayability or root cause. Third, blind environment reuse can improve throughput while increasing contamination-induced failures. The rest of the paper designs RolloutOS around these observations.

4 Design Goals

RolloutOS targets five design goals.

G1: Failure-native execution. Runtime failures must be represented as structured objects with operational semantics, not unstructured exception strings. The runtime must distinguish policy-attributable invalid actions from tool timeouts, environment crashes, scheduler cancellations, and context-limit failures.

G2: Replayable provenance. Every rollout should produce an append-only event stream with sufficient metadata for exact replay when possible and state-equivalent replay otherwise. The trace should contain model, tokenizer, tool,

Trainer / Policy Optimizer
PPO, GRPO, RLOO, or other Agent RL objective
↓ rollout requests ↑ validated trajectories
RolloutOS Runtime
scheduler — session manager — failure monitor — event log — replay engine
↓ actions, leases ↑ observations, rewards, failures
Workers, Tools, and Environments
browsers, web tasks, code sandboxes, APIs, memory stores

Figure 1. RolloutOS treats Agent RL rollout production as a managed systems workload between the optimizer and stateful environments.

reward, and environment provenance.

G3: Environment lifecycle safety. The runtime should treat environment state as a managed resource. It must support reset, health check, snapshot metadata, reuse policy, and contamination detection.

G4: Useful-yield scheduling. The scheduler should optimize useful trajectories per resource-hour. It should consider failure rate, task cost, worker capacity, reward variance, policy lag, and environment health.

G5: Optimizer independence. The system should serve existing Agent RL optimizers. It should export trajectories and failure metadata without requiring a new RL objective.

5 System Overview

Figure 1 shows the RolloutOS architecture. Trainers submit rollout requests to the runtime. The scheduler assigns requests to workers and environments. Each worker runs a managed session, invokes the policy and tools through wrappers, records events, detects failures, and emits trajectory artifacts. The replay engine can later re-execute or inspect traces under exact, model-substituted, or environment-substituted modes.

6 Failure Semantics

Table 1 shows the initial failure taxonomy. Each failure type maps to semantics used by the runtime and downstream training export. For example, an invalid action is attributable to the policy and can produce a training signal, while an environment crash requires reset and should not penalize the policy.

The important design choice is that semantics are explicit and inspectable. This prevents the rollout layer from treating all failures as equivalent. It also avoids penalizing a policy for failures caused by infrastructure or environment nondeterminism.

Failure	Policy	Retry	Salvage	Reset
Invalid action	yes	no	yes	no
Tool timeout	no	yes	yes	no
Tool execution error	mixed	yes	yes	no
Environment crash	no	yes	no	yes
Environment contamination	no	yes	no	yes
Context limit	yes	no	yes	no
Repetitive loop	yes	no	yes	no
No progress	yes	no	yes	no
Rate limit	no	yes	yes	no
Scheduler cancelled	no	yes	yes	no

Table 1. Failure semantics guide retry, replay, cleanup, scheduling, and training export.

7 Event-Sourced Traces

RolloutOS records each trajectory as a JSONL event stream. Each event has a unique identifier, parent identifier, timestamp, session identifier, task identifier, optional sample identifier, optional model and tokenizer metadata, optional environment identifier, payload, token-level metadata, and optional failure record.

Representative event types include session start, model request, model response, action proposal, tool validation, tool execution, observation return, reward emission, environment snapshot, failure detection, session cancellation, and session completion.

This representation supports three replay modes. Exact replay reuses the same model and environment state when all provenance is available. Model-substituted replay fixes the environment and action history while replacing model calls, useful for debugging whether a stronger policy would recover. Environment-substituted replay fixes the model/action stream while using a fresh environment adapter, useful for detecting environment drift or nondeterminism.

8 Session and Environment Lifecycle

The session manager enforces per-rollout budgets and lifecycle transitions. A session starts in a created state, enters running after resource acquisition, and exits as completed, failed, or cancelled. Watchdogs detect excessive turns, wall-clock budget exhaustion, repeated actions, no-progress interaction, tool timeout, and worker cancellation.

Environment lifecycle is managed separately from session state. An environment lease records which worker owns the state, which snapshot or reset operation produced it, and whether the state is healthy for reuse. Health checks validate that expected invariant state holds before reuse. Contamination checks detect mutations that invalidate future samples, such as stale browser state, dirty filesystem state, unexpected database rows, or leftover authentication.

[TODO: Implement and describe concrete environment adapter API: reset, snapshot, restore, health_check, contamination_check, step, close.]

9 USEFUL-YIELD SCHEDULING

Capacity-only schedulers maximize the number of assigned rollout requests but ignore whether those requests are likely to produce useful trajectories. RolloutOS instead scores each candidate request using task statistics:

$$\begin{aligned} score(r,t) &= priority(r) + \alpha \cdot reward\_var(t) \\ &\quad - \beta \cdot failure\_rate(t) - \gamma \cdot cost(r,t) \\ &\quad - \delta \cdot policy\_lag(t). \end{aligned} \tag{1}$$

This score is intentionally simple. The goal is not to introduce a new scheduling algorithm, but to expose failure and cost signals that capacity-only systems ignore. The evaluation compares this policy against FIFO, capacity-only, retry-only, and cost-only baselines.

10 IMPLEMENTATION

We implement a prototype in Python 3.12. The current codebase contains five core modules. `failures.py` defines the failure taxonomy and semantics. `events.py` defines the event model and JSONL persistence. `session.py` implements the session runtime and watchdogs. `replay.py` generates replayability reports from traces. `scheduler.py` implements the initial failure-aware scheduler. The package exposes a small command-line interface for inspecting traces.

[TODO: Expand implementation once runner, FailureBench, replay execution, metrics collector, and environment adapters are implemented.]

11 EVALUATION

We evaluate RolloutOS around four questions.

Q1: Does failure-aware execution detect runtime failures earlier and more accurately? On FailureBench, we compare timeout-only and message-only baselines against RolloutOS. Metrics are precision, recall, macro F1 by failure type, mean time to detection, and turns wasted after oracle failure.

Q2: Do event-sourced traces improve replay and debugging? We compare raw stdout/stderr, message-only traces, and RolloutOS event traces. Metrics are replay success rate, root-cause attribution accuracy, actionable diagnosis rate, and time-to-root-cause.

System	Episodes	Macro F1	Acc.	Wasted Turns	Failed Cost	Replayable	Useful/Cost
raw_timeout	40	0.000	0.000	6.25	320.0	0.000	0.000
message_trace	40	0.222	0.250	0.75	105.0	0.250	0.095
retry_only	40	0.333	0.375	1.12	139.8	0.375	0.107
rolloutos	40	1.000	1.000	0.00	62.5	0.750	0.480

Table 2. FailureBench detection and resource-cost results. Failed cost is measured in deterministic synthetic resource units, not wall-clock time.

Policy	Success	Reset	Restore	Reuse	Contam. Fail	Cost/Success
recreate	50/50	50	0	0	0	1.100
blind_reuse	1/50	1	0	49	49	6.000
contamination_aware	50/50	1	49	0	0	0.316

Table 3. Synthetic environment lifecycle benchmark. Cost is measured in deterministic reset/restore/step units.

Q3: Does useful-yield scheduling improve rollout production? We compare FIFO, capacity-only, retry-only, and RolloutOS scheduling. Metrics are useful trajectories/hour, failed rollout cost, environment-hours/success, zombie session rate, and tail latency.

Q4: Can environment lifecycle management reduce cost without increasing contamination? We compare recreate-every-episode, blind reuse, fixed-TTL reuse, health-check reuse, and contamination-aware reuse. Metrics are reset latency, environment utilization, contamination rate, contamination-induced failure, and cost per successful trajectory.

11.1 FailureBench Results

Table 2 reports a local reproducible FailureBench run using the configuration in `experiments/configs/local_suite.json`. The benchmark covers eight deterministic failure types with oracle labels. The raw timeout baseline detects only failures that eventually exhaust a generic budget, while message-only and retry-only baselines recover some tool-visible failures but cannot reliably distinguish policy, runtime, and environment causes. RolloutOS detects all oracle failure types in this controlled setting, eliminates wasted turns after the oracle failure point, and increases replayable failed trajectories.

11.2 Environment Lifecycle Results

Table 3 reports a synthetic mutable-environment lifecycle benchmark. Recreating an environment for every episode avoids contamination but pays the full reset cost each time. Blind reuse has low reset cost but allows dirty state to leak into subsequent episodes. The contamination-aware policy restores from a clean snapshot when a health check detects dirty state, preserving all successful episodes while reducing cost-per-success relative to full recreation.

Model	Vocab	Prompts	Mean Tok.	Max Tok.	Drift Valid
Qwen3-4B	151669	3	20.0	23	3/3
Qwen3-8B	151669	3	20.0	23	3/3
Llama-3.1-8B-Instruct	128256	3	46.7	50	3/3

Table 4. Tokenizer provenance and token-drift audit using locally cached open-weight models. Drift Valid counts prompts whose decode/re-encode pass preserves the recorded token IDs.

11.3 Model Provenance and Token Drift Results

Table 4 reports a local tokenizer provenance audit over three cached open-weight instruction models. The audit does not yet measure model generation quality; instead, it validates the token-native trace path required by Agent RL training export. For each model, the runtime loads the local Hugging Face tokenizer, hashes the tokenizer and chat template, renders three agent prompts with the model's chat template, records exact token IDs and loss masks into event traces, and checks whether decode/re-encode round trips preserve token IDs.

The Qwen3-4B and Qwen3-8B checkpoints share the same tokenizer fingerprint and produce identical prompt token counts. Llama-3.1-8B-Instruct uses a different chat template and tokenizer, producing 46.7 mean prompt tokens on the same prompts versus 20.0 for Qwen3. All three models pass the drift validator on all audited prompts. This result closes the first model-facing systems requirement: rollout traces can now carry real model version, tokenizer hash, token IDs, and loss masks rather than only message text.

11.4 Local Model Generation Smoke Results

Table 5 reports the first end-to-end local model generation run. Unlike the tokenizer audit, this experiment loads Qwen3-4B with `AutoModelForCausalLM`, renders the same agent prompts, generates 32 tokens per prompt with greedy decoding, computes transition log-probabilities, and writes both request and response events with `model_version`, `tokenizer_hash`, `token_ids`, `logprobs`, and `loss_mask`. This is still a smoke-scale model experiment rather than a task-success result, but it closes the missing systems path from model execution to token-native rollout traces.

On a local A100, Qwen3-4B generates 96 output tokens for three prompts at 21.54 tokens/s after loading, with mean generated-token log-probability -0.103. The trace contains exact per-token log-probabilities for training export and replay inspection.

11.5 Model-to-Action Contract Results

Table 6 takes one step beyond free-form generation: Qwen3-4B is used as a browser-action policy on three MiniWoB

Model	Prompts	In Tok.	Out Tok.	Load s	Tok/s	Mean Logp
Qwen3-4B	3	60	96	6.6	21.54	-0.103

Table 5. Local Qwen3-4B generation smoke test with token-native event traces. Latency excludes model loading; Load s reports local checkpoint load time.

Model	Protocol	Budget	Success	Parse	Invalid	No Prog.	Tok/s	Mean Logp
Qwen3-4B	default	96	0/9	0/9	9	0	31.14	-0.031
Qwen3-4B	no_thinking	96	9/9	9/9	0	0	34.66	-0.000
Qwen3-4B	guided_json	96	9/9	9/9	0	0	16.62	-0.000
Qwen3-4B	default	384	7/9	7/9	2	0	30.32	-0.038
Qwen3-4B	no_thinking	384	9/9	9/9	0	0	33.69	-0.000

Table 6. Qwen3-4B as a browser-action policy on MiniWoB contract tasks under two token budgets and two action-channel protocols.

contract tasks and three deterministic seeds per task. The runtime renders each browser observation, asks the model for a JSON browser action, parses the output, executes the action in the contract environment, and records request tokens, generated tokens, log-probabilities, proposed actions, tool execution, rewards, and typed failures in one event stream.

This experiment exposed a systems failure mode that is invisible in ordinary task-success reporting. At a 96-token action budget, the default Qwen3 thinking template produces 0/9 parseable actions because the budget is consumed by reasoning text. Increasing the budget to 384 tokens recovers 7/9 successful executions, but it emits 3096 generated tokens for the nine episodes and still leaves two invalid-action failures. The thinking-disabled structured action protocol solves 9/9 episodes at both budgets while emitting only 727 generated tokens. A local JSON-schema constrained decoder also solves 9/9 episodes and removes parser failures, but reduces throughput from 34.66 to 16.62 generated tokens/s on this workload. This supports a concrete runtime requirement: rollout workers must control chat-template, decoding constraints, and token budget per action channel, not simply call a model endpoint and hope for a valid tool call.

We also feed the measured action-channel summaries into the worker-pool simulator, so the scheduling workload reflects real Qwen3-4B parse/success rates and measured per-action latency rather than only hand-written synthetic classes. Table 7 reports the resulting system consequence. On this empirical action-channel workload, retry-only increases attempts from 200 to 245, raises failed cost, and reduces useful throughput. Failure-aware execution does not claim additional task successes; it converts typed invalid-action failures into replayable useful artifacts, raising useful trajectories/hour from 4646.8 to 5995.8 while preserving the same successful trajectory rate.

Policy	Useful/hr	Succ./hr	Failed Cost	Zombie	P95 Lat.	Util.	Attempts
model_action_fifo	4646.8	4646.8	180.1	0.000	11.3	1.000	200
model_action_retry_only	3793.8	3793.8	338.4	0.008	11.3	0.961	245
model_action_failure_aware	5995.8	4646.8	180.1	0.000	11.3	1.000	200

Table 7. Worker-pool throughput simulation parameterized by the measured Qwen3-4B model-to-action summaries in Table 6.

Policy	Useful/hr	Succ./hr	Failed Cost	Zombie	P95 Lat.	Util.	Attempts
fifo	1750.5	1750.5	1289.0	0.071	36.0	0.942	240
retry_only	1581.8	1581.8	1561.0	0.071	24.0	0.939	281
failure_aware	8037.2	4437.2	307.0	0.000	6.0	0.959	240

Table 8. Deterministic worker-pool throughput simulation. Useful trajectories include successful rollouts and replayable typed failures.

11.6 Worker-Pool Throughput Results

Table 8 reports a deterministic worker-pool simulation with 240 rollout requests across five workload classes and eight workers. The workload mixes short successful browser tasks, stale-DOM invalid actions, wait-loop timeouts, retryable tool timeouts, and contaminated environments. FIFO treats failed sessions as ordinary long-running work. Retry-only adds attempts for retryable failures but does not reduce wasted runtime cost. The failure-aware policy uses typed failure semantics to terminate known-bad rollouts early and treat replayable failures as useful debugging or training artifacts.

In this setting, failure-aware scheduling increases useful trajectories/hour from 1750.5 to 8037.2, reduces failed rollout cost from 1289.0 to 307.0 deterministic units, eliminates zombie sessions, and reduces P95 latency from 36.0 to 6.0 units. The successful trajectory rate also improves because workers stop being occupied by late-detected failures.

11.7 MiniWoB Browser Contract Results

Table 9 reports a deterministic MiniWoB browser contract run over the fixed 20-task subset and five development seeds per task. This is not yet the live browser experiment; it fixes the runtime contract that the live adapter must satisfy: actions carry a DOM hash, selectors are validated against actionable elements, wait loops produce no-progress failures, and every episode emits a replayable event trace. The oracle policy solves all 100 episodes. Stale-DOM and invalid-selector policies fail immediately with typed policy-attributable invalid-action records. The wait-loop policy fails all 100 episodes through the runtime no-progress watchdog after two turns.

[TODO: Extend Table 2 to the full test split and add real browser/tool workloads.]

Policy	Success	Succ. Rate	Replayable	Turns	Invalid	Loop	No Prog.
oracle	100/100	1.000	1.000	1.00	0	0	0
stale_dom	0/100	0.000	1.000	1.00	100	0	0
invalid_selector	0/100	0.000	1.000	1.00	100	0	0
wait_loop	0/100	0.000	1.000	2.00	0	0	100
repeated_action	60/100	0.600	1.000	1.40	0	0	40

Table 9. MiniWoB browser contract results on 20 tasks $\times$ 5 seeds. The contract harness quantifies browser-runtime failure handling before attaching the same adapter interface to live MiniWoB++ pages.

Experiment	**Workload**	**Baselines**	**Primary metric**
Failure detection	FailureBench	timeout, message trace	macro F1
Replay/debugging	FailureBench, MiniWoB	raw, message trace	replay success
Scheduling	FailureBench, WebShop	FIFO, capacity, retry	useful traj/hr
Lifecycle	MiniWoB, SWE-bench	recreate, blind reuse	cost/success

Table 10. Evaluation plan for a systems-focused MLSys submission.

12 Discussion

Relationship to RL algorithms. RolloutOS does not claim that failure semantics alone solve policy learning. Instead, it provides cleaner inputs to existing optimizers and more reliable rollout production. Training improvements, when present, should be interpreted as downstream effects of better systems behavior: fewer invalid samples, better partial trajectory salvage, and less environment waste.

Failure semantics can bias learning. Structured failures can introduce reward hacking if negative rewards are assigned naively. RolloutOS therefore separates runtime semantics from optimizer policy. The runtime records whether a failure is training-relevant; the training exporter decides how to convert that signal.

Replay is conditional. Exact replay is only possible when the model, tokenizer, environment state, tool versions, and external responses are controlled. The system therefore reports determinism levels rather than promising universal exact replay.

13 Related Work

[TODO: Expand related work with complete author metadata before submission. MLSys 2026 requires each reference to list all authors explicitly.] Agent RL frameworks motivate the need for high-throughput multi-turn rollout production (Zhang et al., 2025; NVIDIA Research, 2026). Agent-system reliability studies identify scheduling, context degradation, tool execution, and lifecycle bugs as recurring failure modes (Anonymous, 2026a;b; 2025). Structured-output serving systems such as OpenAI Structured Outputs, vLLM guided decoding, and SGLang structured decoding show that reliable JSON and tool actions are runtime constraints rather than prompt conventions (OpenAI, 2024;

vLLM Project, 2026; Zheng et al., 2023). Agent runtime systems such as OpenAI Agents and LangGraph expose tracing, persistence, and checkpointing as first-class execution mechanisms (OpenAI, 2026; LangChain, 2026). RolloutOS is complementary to these efforts: it provides a rollout runtime substrate and measurement framework rather than a new agent policy or reward model.

14 Conclusion

Long-horizon Agent RL turns rollout production into a systems problem. Failures, replayability, environment lifecycle, and scheduling decisions directly affect the cost and quality of training data. RolloutOS addresses this gap by treating rollouts as managed runtime sessions with typed failure semantics, event-sourced traces, replay support, environment lifecycle management, and useful-yield scheduling. The resulting system is designed to make failed rollouts observable, attributable, replayable when possible, and useful to scheduling and debugging.

References

Anonymous. Where llm agents fail and how they can learn from failures, 2025. URL https://arxiv.org/abs/2509.25370. TODO: replace with complete author list before MLSys submission.

Anonymous. Agentrm: An os-inspired resource manager for llm agent systems, 2026a. URL https://arxiv.org/abs/2603.13110. TODO: replace with complete author list before MLSys submission.

Anonymous. An empirical study of bugs in modern llm agent frameworks, 2026b. URL https://arxiv.org/abs/2602.21806. TODO: replace with complete author list before MLSys submission.

LangChain. Langgraph persistence documentation, 2026. URL https://langchain-ai.github.io/langgraph/concepts/persistence/. Accessed 2026-05-20.

NVIDIA Research. Prorl agent: Rollout-as-a-service for rl training of multi-turn llm agents, 2026. URL https://arxiv.org/abs/2603.18815. TODO: replace with complete author list before MLSys submission.

OpenAI. Introducing structured outputs in the api, 2024. URL https://openai.com/index/introducing-structured-outputs-in-the-api/. Accessed 2026-05-20.

OpenAI. Openai agents sdk tracing documentation, 2026. URL https://openai.github.io/openai-agents-python/tracing/. Accessed 2026-05-20.

vLLM Project. vllm structured outputs and guided decoding documentation, 2026. URL https://docs.vllm.ai/. Accessed 2026-05-20.

Zhang, H., Liu, X., Lv, B., Sun, X., Jing, B., Iong, I. L., Hou, Z., Qi, Z., Lai, H., Xu, Y., Lu, R., Wang, H., Tang, J., and Dong, Y. Agentrl: Scaling agentic reinforcement learning with a multi-turn, multi-task framework, 2025. URL https://arxiv.org/abs/2510.04206.

Zheng, L., Yin, L., Xie, Z., Sun, J., Huang, C., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Efficiently programming large language models using sglang, 2023. URL https://arxiv.org/abs/2312.07104.